

A DOMAIN-DRIVEN AGENTIC WORKFLOW

Agentheim

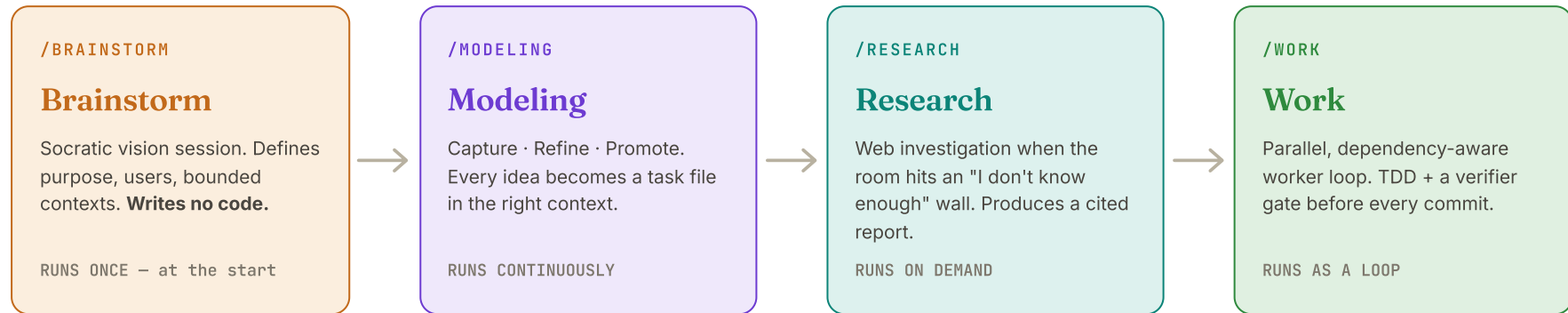
Bounded contexts and ubiquitous language — from a thought to *committed code*.

Agentheim makes **domain-driven design** the spine of the whole workflow: every idea is modeled into the right **bounded context** and described in the domain's own **ubiquitous language** before any code is written. Four skills, one orchestrator, and a handful of specialist agents keep the model and the code in sync.

■ Brainstorm ■ Modeling ■ Research ■ Work ■ Orchestrator & specialists

Four skills, one direction of travel

Brainstorm runs once to give the project a shape. **Modeling** then runs continuously, turning thoughts into refined tasks. **Research** is pulled in on demand whenever knowledge runs out. **Work** loops over the ready tasks and ships them.



Work can spawn new backlog items · Modeling keeps feeding todo · the loop never formally "ends"

The knowledge layer everything reads & writes

vision.md

context-map.md

contexts/<bc>/

knowledge/decisions/ — ADRs

knowledge/research/

knowledge/protocol.md

INDEX.md ×N

Markdown only — reviewable before any code runs

Brainstorm — finding the shape

A Socratic sparring session, not a scribe. The user leaves feeling they *discovered* the shape of the thing. The hard rule: **no code** — every output is reviewable markdown.

/BRAINSTORM

Socratic Vision Session

Asks one question at a time across eight dimensions — purpose, users, the problem, the domain language, success, non-goals, constraints, and the seeds of bounded contexts.

Triggers on: "start a project", "create a vision", "build X"

Default mode: Interrogator

Produces: markdown, never code

1

The Socratic loop

Probe abstractions ("walk me through Alice's Tuesday"), surface conflations, question certainty. Stop when both sides describe the system in the same language.

2

Write vision.md

Purpose · Users · The problem · What success looks like · Non-goals · Ubiquitous language seed · Open questions.

3

Write context-map.md — if warranted

Only when the domain has more than one area of concern. Names the bounded contexts, classifies them core / supporting / generic, maps their relationships.

4

Architecture foundation pass

The one structured exception to "no code is queued yet". Unconditionally creates the `infrastructure/` BC, then via the orchestrator runs the **architect** to surface load-bearing tech decisions.

5

Emit foundation tasks

One type: decision task per area (global → infrastructure BC, BC-local → originating BC), one type: spike walking-skeleton, and a styleguide task if the vision implies any UI.

The walking skeleton is the project's first prototype — feature-thin, architecture-thick. It depends on every decision task, so the foundations are settled before the first real feature lands.

Modeling — the main entry point for ideas

Every thought from here on enters through **Modeling** — a one-line bug, a feature request, a refinement, a whole new subsystem. It does three things, and it routes deep thinking to the orchestrator.

/MODELING

Capture • Refine • Promote

Decides which of three actions applies, routes the idea to the right bounded context, and writes a task file with auto-populated backlinks.

Triggers on: "I have an idea",
"there's a bug", "refine X",
"promote X to todo"

Capture mode: Facilitator
Refine mode: Interrogator

CAPTURE

An idea arrives. Route it to a BC, run a prior-art lookup, decide refinement level.

→ backlog/ or todo/

REFINE

Deepen a backlog item via the orchestrator. Split it, sharpen acceptance criteria, pull in research.

→ stays backlog/ or → todo/

PROMOTE

Readiness check, then move. Concrete criteria, clear scope, dependencies known.

→ backlog/ → todo/



Routing: global vs BC-local

The test — "If this BC didn't exist, would the change still need to happen?" Yes → infrastructure/. No → the originating BC.



Backlink lookup

Before writing the task, the matcher scans BC and global indexes. It auto-populates `related_adrs`, `related_research`, and `prior_art` — so workers never arrive in isolation.



Delegate the hard thinking

For anything non-trivial, Model hands off to the **orchestrator**, which routes to strategic / tactical / architect / research specialists and returns a refined task.

Task types carried in frontmatter: feature • bug • refactor • chore • spike • decision. A decision task's deliverable is an ADR, not code — but it still flows through the same backlog discipline.

Six stances — same artifact, different conversation

Modes apply only to **Brainstorm** and the conversational parts of **Modeling** (Capture & Refine). They change the *path* to the artifact, never its format. The user switches them mid-session — "switch to challenger", "synthesize what we have".

Interrogator

DEFAULT · BRAINSTORM + REFINE

Naive, curious, critical questions — one at a time.
The classic Socratic stance.

"What would have to be true for this to be wrong?"

Suggestor

FOR STUCK ROOMS

Makes smart assumptions and concrete proposals.
The user accepts, rejects, or refines.

"Sounds like you want X. Push back if your domain breaks that."

Challenger

USE SPARINGLY

Adversarial skeptic. Plays a stakeholder who isn't buying it — presses the weakest part.

"This breaks the moment a user has two accounts. Convince me."

Storyteller

WHEN IT'S GONE ABSTRACT

Narrates concrete scenarios with named characters to pull the conversation down to specifics.

"Tuesday morning. Alice opens the app before her coffee's done..."

Facilitator

DEFAULT · MODELING CAPTURE

Scribe stance. Minimal interjection — the humans drive, the skill captures and structures.

"I'm hearing three points — A, B, C. Which do we go deeper on?"

Synthesizer

PERIODIC SWITCH-INTO

Reflects back tensions, contradictions, and emergent themes. Best after a long stretch.

"Two things don't quite fit: X from earlier, Y just now. How do they relate?"

Research — how outside knowledge enters

Research is about *understanding*, not deciding. It produces a cited report — never an ADR. Decisions come later, in Modeling or Work, and they *cite* the report.

/RESEARCH

Web Investigation, Written Down

The **researcher** agent runs a breadth pass, fetches the best sources, cross-checks claims, and writes a structured report.

Triggers: "research X", "what's the state of the art", or internally when brainstorm / modeling / work hit a knowledge wall

Output: knowledge/research/

1

Breadth, then depth

WebSearch for a wide pass, then WebFetch on the most promising 3–7 sources. Independent angles run in parallel.

2

Cross-check & write

Single-source claims are flagged as such; disagreements between sources are named, not hidden. Report lands at knowledge/research/<slug>-<date>.md.

3

Link back — bidirectionally

The task's Notes get a link, the task's related_research frontmatter gets the slug (so workers pre-load it), and the report's related_tasks gets the task id.

4

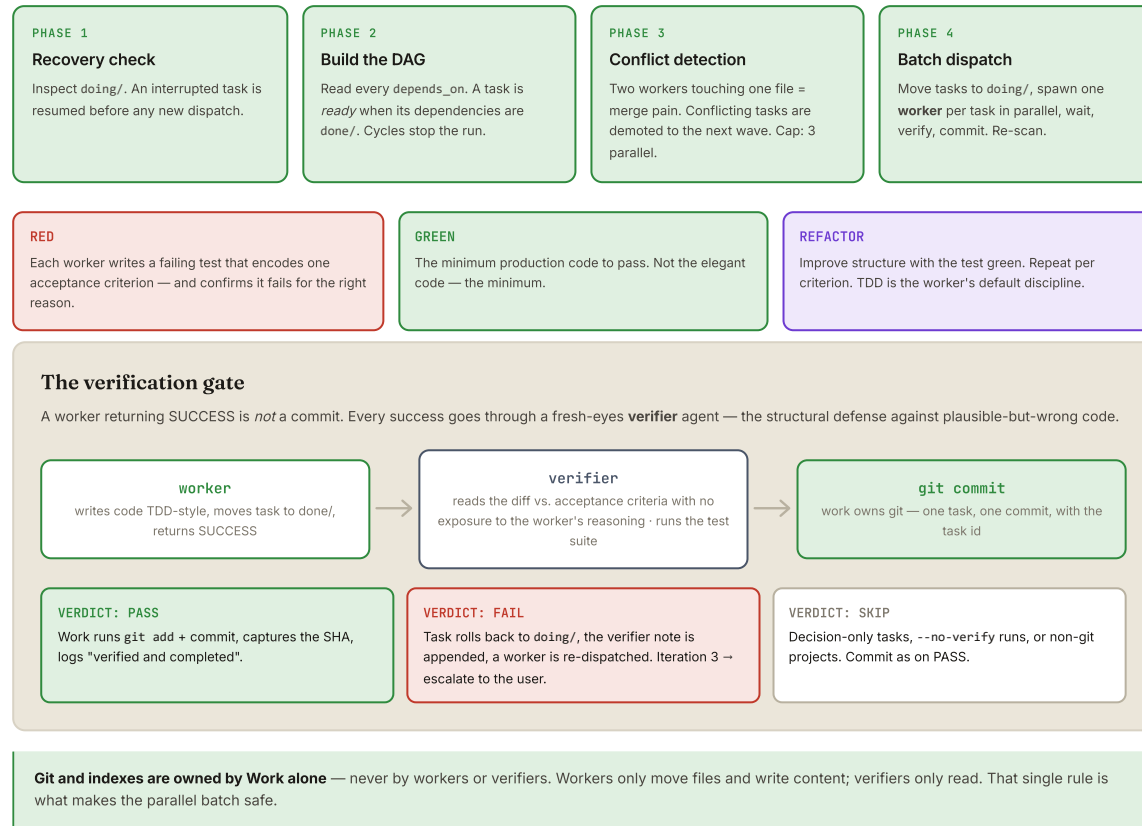
Index & log

BC-scoped reports go in the BC index; cross-cutting ones go global. A protocol entry records who requested it.

The bidirectional link is the whole point. It is how a research finding stops being a dead document and becomes something a worker arrives already knowing — without re-deriving it.

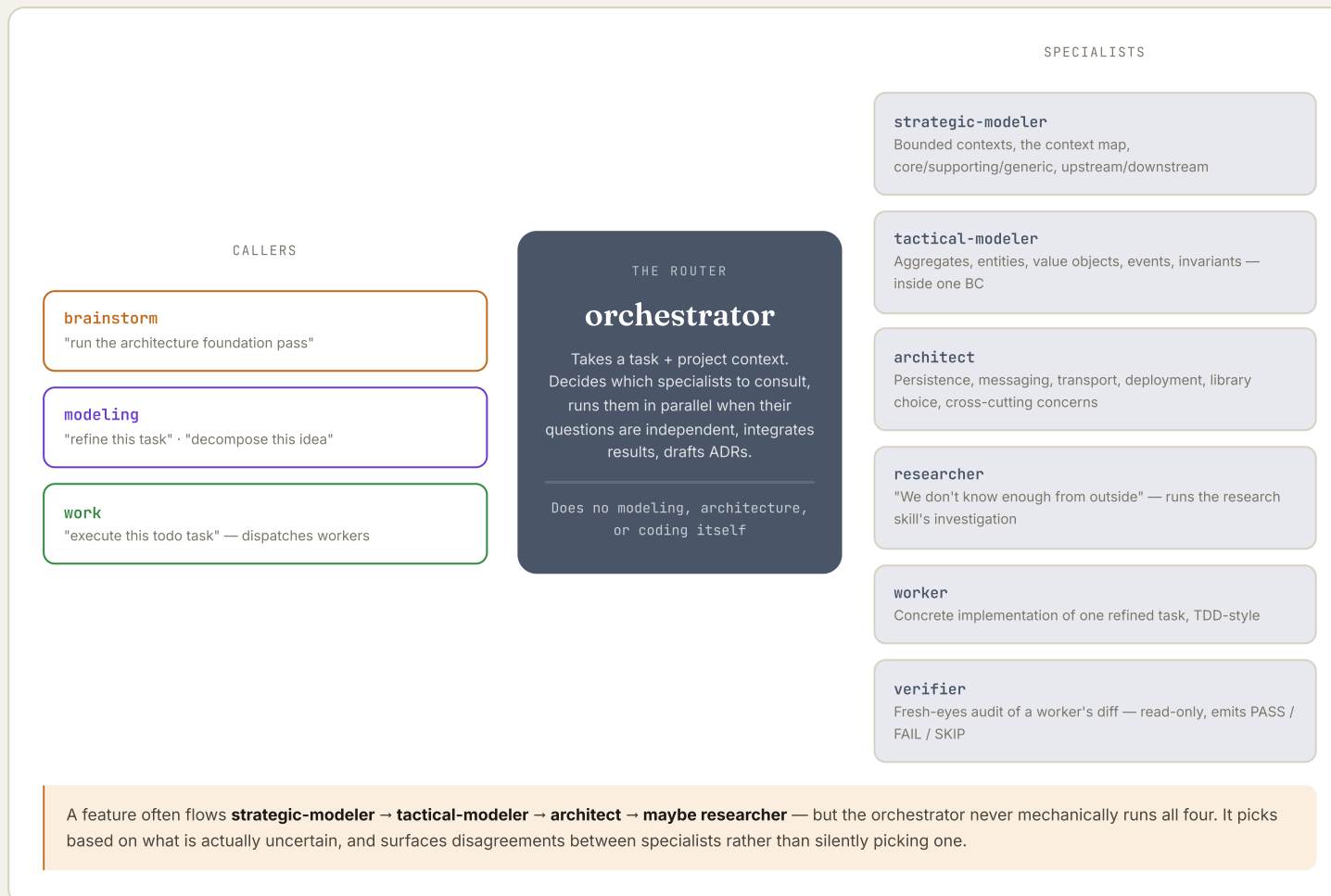
Work — the parallel, verified loop

Work turns refined todo/ tasks into real code. The orchestrating skill **never writes code** — it scans, builds the dependency graph, dispatches workers, and owns git. Staying lean is what lets it run long batches without exhausting context.



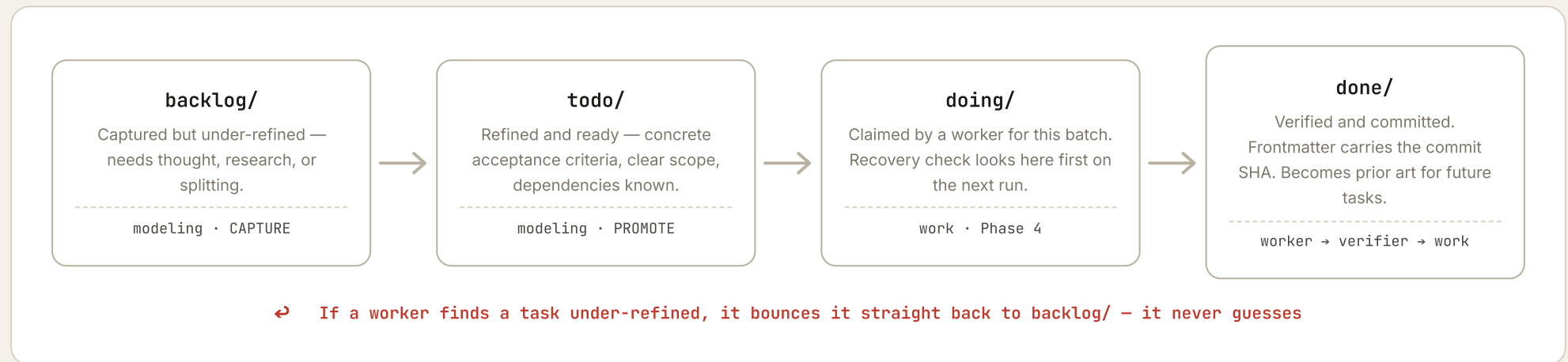
The orchestrator routes — it does not decide

Skills don't talk to specialists directly. They hand a question to the **orchestrator**, which picks which specialist(s) should answer, runs them (in parallel when independent), resolves conflicts, and returns something concrete the caller can act on.



One task, four folders

Every idea becomes a markdown file that physically moves between directories inside its bounded context. The folder *is* the status — and which actor is allowed to move it is fixed.



A bug that needs research

The clearest way to see the whole rail in motion: you find a bug, it can't be fixed without outside knowledge, and that knowledge has to reach the worker who fixes it.

1

MODELING · CAPTURE

The bug is captured

You say "there's a bug: X happens when Y." Modeling routes it to the right bounded context, sets `type: bug` in the frontmatter, and — because it's not yet clear how to fix — files it under `backlog/`.

```
contexts/<bc>/backlog/<bc>—014-bug.md · related_research: []
```

2

MODELING · REFINE

Refinement hits a knowledge wall

You refine the bug. Modeling delegates depth to the orchestrator — and the orchestrator (or you) hits an "we don't know enough about X" wall. That is the explicit trigger for Research.

3

RESEARCH

Research runs in parallel

The researcher agent investigates while modeling continues on whatever doesn't depend on the answer. It writes a cited report to the knowledge layer.

```
knowledge/research/<slug>—2026-05-14.md
```

4

RESEARCH · LINK BACK

The findings attach to the task

Bidirectional linking fires: the task's Notes get a link, its `related_research` frontmatter gets the report slug, and the report's `related_tasks` gets the bug's id. The research is now *part of the task*, not a document beside it.

```
<bc>—014-bug.md · related_research: [<slug>—2026-05-14]
```

5

MODELING · REFINE → PROMOTE

The description gets sharper, then promoted

With the findings folded in, Refine rewrites the bug — concrete root cause in Notes, testable acceptance criteria. Now that it's genuinely ready, it's promoted to `todo/`.

```
contexts/<bc>/todo/<bc>—014-bug.md
```

6

WORK · WORKER

The worker arrives already knowing

Work picks the bug up. The worker pre-loads `related_research` from the frontmatter — it starts the fix already holding the research conclusion instead of re-deriving it. TDD turns the bug repro into a failing test first, then green.

7

WORK · VERIFIER → COMMIT

Verified, then committed

The verifier audits the diff against the acceptance criteria with fresh context and runs the suite. On PASS, Work commits — one task, one commit — and the bug file lands in `done/`, now itself prior art for the next related task.

```
bug(<bc>): fix X when Y [<bc>—014]
```

The shared memory under everything

All four skills and every agent read and write the same `.agentheim/` tree. It is what keeps the workflow coherent across sessions — and it is all reviewable markdown.

`vision.md`

Purpose, users, the problem, success, non-goals, ubiquitous language seed. Written by brainstorm.

`context-map.md`

The bounded contexts, their classification, and the relationships between them.

`contexts/<bc>/`

Per-context home: README (ubiquitous language), INDEX, and the backlog / todo / doing / done folders.

`knowledge/decisions/`

ADRs — scoped global or to a BC. Written by the orchestrator and workers, committed by work.

`knowledge/research/`

Cited research reports. Linked bidirectionally to the tasks and ADRs that cite them.

`knowledge/protocol.md`

The project's chronological diary — every capture, refine, batch, commit, and bounce, newest on top.

`knowledge/index.md`

Top-level catalog: current BCs, global ADRs, cross-BC research. The map skills read first.

`contexts/<bc>/INDEX.md`

Per-BC catalog of tasks, ADRs, research and concepts — kept current by modeling and work.

`contexts/<bc>/concepts/`

Opt-in synthesis pages — created only when work spots several artifacts converging on one concept.

A window onto the knowledge layer

Everything above is reviewable markdown on disk. The **dashboard** is an optional local web UI over that same `.agentheim/` tree — read-mostly, live, and never a second source of truth. It is driven by `/dashboard`: the *single, deliberate* exception to the "phrasing, not slash commands" rule, because launching a server is a process action, not a Socratic dialogue.

`/DASHBOARD`

Launch · Stop · Status

A thin slash-command trigger over one cross-platform launcher (`dashboard/launch.mjs`). The server is Node-stdlib only — no framework, no install step.

Surface: the one slash command (not a skill, not phrase-triggered)

Writes: one — drag `backlog→todo` to Promote, via the shared mover



`/dashboard`

Launches (or reuses) the detached server and auto-opens the browser at `http://127.0.0.1:<port>/`. Relaunch is idempotent — a live runfile is reused, a stale one reaped.



`/dashboard stop`

Stops the detached server and removes the runfile.



`/dashboard status`

Reports whether it's running, and on which port — read-only, never launching or stopping.

BOARD

A flat Kanban pooling every BC's tasks across the four lifecycle columns; each card carries its context.

`backlog · todo · doing · done`

SLIDE-OVER

One universal drawer renders any artifact — task, BC README, vision, context map, ADR, research — as markdown.

`→ /api/doc, client-rendered`

LIVE

An SSE stream re-fetches the tree whenever a skill moves a file on disk, so the board stays current.

`→ /api/events (push)`

Disk stays the source of truth. The board is a projection rebuilt from `/api/tree`; its one write — Promote — goes through the same `applyTaskMove` the skills use, never a UI-only path.

Agentheim — measure twice, cut once.

Brainstorm finds the shape · Modeling turns thoughts into tasks · Research fills the gaps · Work ships verified code. The orchestrator routes; the specialists think; the knowledge layer remembers.